SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA5116
COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

Total Marks: 50

Annexure A

Code of Conduct:

I U.Divya/192372277 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Divya

Annexure A

1. Debugging Hash Function Implementation (SHA Family)

ntroduction

SHA (Secure Hash Algorithm) is used to generate a fixed-size hash value from input data. If the same input produces different hash values, the implementation contains errors.

1. Identify Errors

- Incorrect message padding.
- Wrong block size (should be 512 bits for SHA-256).
- Improper message length calculation.
- Incorrect bitwise operations.
- Endianness mismatch (little-endian instead of big-endian).

2. Fix Message Padding

- Append 1 bit after the message.
- Add 0 bits until message length becomes 448 mod 512.
- Append the original message length as a 64-bit intege**r.**

3. Fix Block Processing

- Divide the message into 512-bit blocks.
- Process each block sequentially.
- Update intermediate hash values correctly after every block.

4. Fix Bitwise Operations

Use correct rotation functions.

Example:

```
def right_rotate(x, n):
    return ((x >> n) | (x << (32 - n))) & 0xFFFFFFFF
```

5. Correct Endianness

Read data using big-endian format:

int.from_bytes(data, "big")

6. Performance Optimization

- Process files in chunks (64 KB or larger).
- Avoid loading entire files into memory.
- Reuse buffers during hashing.
- Use optimized libraries like hashlib.

7. Result

The same input always generates the same SHA hash.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

Problem

The SSL/TLS handshake is slow and occasionally fails.

Issues Identified

- Invalid certificate verification.
- Incorrect public/private key exchange.
- Improper cipher suite negotiation.
- Session keys recreated every connection.

Fixes

- Verify certificate chain and expiration date.
- Validate server identity.
- Use secure key exchange such as ECDHE.
- Handle handshake exceptions properly.

Optimization

- Enable TLS Session Resumption.
- Cache session tickets.
- Reuse established session keys.
- Reduce unnecessary certificate verification.

Conclusion

Proper certificate validation and session resumption significantly improve TLS performance without sacrificing security.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

Problem

Generated signatures fail during verification.

Issues Identified

- Invalid prime parameters (p, q, g).
- Weak or repeated random nonce (k).
- Incorrect modular inverse calculations.
- Errors in verification equations.

Fixes

- Generate valid DSA parameters.
- Use cryptographically secure random numbers.
- Compute modular inverse correctly.
- Validate signature values before verification**.**

Optimization

- Use fast modular exponentiation.
- Apply efficient modular arithmetic.
- Reuse precomputed values when possible.
- Use optimized Python cryptographic libraries.

Randomness Impact

Weak randomness can:

- Reveal the private key.
- Produce duplicate signatures.
- Enable signature forgery.
- Completely compromise DSA security.

Conclusion

Secure parameter generation and high-quality randomness ensure valid and secure digital signatures.

4. Debugging Key Management System

Problem

Cryptographic keys are stored insecurely and retrieval fails.

Issues Identified

- Keys stored in plaintext.
- Weak encryption.
- No key rotation.
- Poor access control.
- Improper backup procedures.

Fixes

- Encrypt stored keys using AES-256.
- Store master keys separately.
- Implement secure key generation.
- Enable key rotation.
- Apply role-based access control.

Optimization

- Use Hardware Security Modules (HSM) or secure key vaults.
- Encrypt key databases.
- Maintain audit logs.
- Cache decrypted keys only temporarily.
- Security Analysis

Poor key management can:

- Leak confidential data.
- Allow unauthorized decryption.
- Break authentication systems.
- Compromise the entire cryptographic infrastructure.

Conclusion

Strong encryption, proper lifecycle management, and secure storage protect cryptographic keys effectively.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

Problem

The secure file transfer system experiences slow transfers and data corruption.

Issues Identified

- Encryption and decryption become unsynchronized.
- Packet ordering problems.
- Missing integrity verification.
- Large file buffering delays.

Fixes

- Synchronize encryption and decryption states.
- Use packet sequence numbers.
- Verify packets using SHA-256 or HMAC.
- Retransmit corrupted packets.

Optimization

- Transfer files in chunks.
- Enable parallel packet processing.
- Compress data before encryption.
- Use streaming encryption for files larger than 100 MB.

Conclusion

Optimized packet handling, integrity verification, and chunk-based transfer improve both speed and reliability while maintaining strong security.